

ACADEMIC PROJECT REPORT

SHIVANGI MOBILE — OMNICHANNEL E-COMMERCE & PRE-OWNED DEVICE RETAIL PLATFORM

A Full-Stack Progressive Web Application (PWA) with Secure REST API, Cloudinary CDN, and MongoDB Atlas Integration

A Capstone / Final Year Project Report Submitted in Partial Fulfillment of the Requirements for the Degree of
Bachelor of Technology (B.Tech) / Bachelor of Computer Applications (BCA) / Master of Computer Applications (MCA)
In Computer Science & Engineering / Information Technology

Submitted By:

- **Candidate Name:** [Student Name / Kumar]
- **Roll Number / Enrollment No:** [Your Roll Number]
- **Department:** Department of Computer Science & Engineering
- **Institution / College:** [Your College / Institute Name]
- **Academic Session:** 2025 – 2026

Under the Guidance of:

- **Project Supervisor / Guide:** [Guide Name / Faculty Designation]
 - **Department:** Department of Computer Science & Engineering
-

TABLE OF CONTENTS

1. [Certificate of Completion](#)
2. [Candidate Declaration](#)
3. [Acknowledgement](#)
4. [Abstract](#)
5. [Chapter 1: Introduction & Problem Statement](#)
 - 1.1 Background & Motivation
 - 1.2 Limitations of Traditional Brick-and-Mortar Retail
 - 1.3 Proposed System & Objectives
 - 1.4 Scope of the Project
6. [Chapter 2: System Requirements Specification \(SRS\)](#)
 - 2.1 Hardware Requirements
 - 2.2 Software & Tool Requirements
 - 2.3 Functional Requirements
 - 2.4 Non-Functional Requirements
7. [Chapter 3: System Design & Architecture](#)
 - 3.1 Architectural Overview
 - 3.2 Data Flow Diagrams (DFD Level 0 & Level 1)
 - 3.3 Use Case Diagram & Actor Model
 - 3.4 Entity-Relationship (ER) Schema & Data Models
 - 3.5 Order Processing Sequence Diagram
 - 3.6 Order Lifecycle State Transition Diagram
8. [Chapter 4: Technology Stack Justification](#)
 - 4.1 Frontend Technologies
 - 4.2 Backend & API Technologies
 - 4.3 Database & Cloud Storage
 - 4.4 Authentication & PWA Infrastructure
9. [Chapter 5: Detailed Module Implementation](#)
 - 5.1 Storefront & Interactive Catalog Module
 - 5.2 Certified Pre-Owned / Refurbished Device Module
 - 5.3 Cart, Wishlist & Device Comparison Module
 - 5.4 Dynamic UPI QR & COD Checkout Module

- 5.5 Omnichannel WhatsApp Ordering Module
 - 5.6 Interactive Store Locator Module (Leaflet OSM)
 - 5.7 Progressive Web App (PWA) Engine
 - 5.8 Administrative ERP Portal Module
10. [Chapter 6: Security & Data Integrity Measures](#)
 - 6.1 Server-Side Price Cross-Validation (SEC-004)
 - 6.2 Stateless JWT & Bcrypt Hashing (SEC-003)
 - 6.3 HTTP Security Headers via Helmet (SEC-010)
 - 6.4 Strict CORS Origin Allowlist (SEC-007)
 - 6.5 Rate Limiting & DoS Protection (SEC-008)
 - 6.6 NoSQL Injection Sanitization (SEC-014)
 11. [Chapter 7: REST API Specification](#)
 12. [Chapter 8: Testing & Quality Assurance](#)
 - 8.1 Unit & Integration Testing
 - 8.2 Security Testing & Vulnerability Assessment
 - 8.3 Performance & Responsiveness Testing
 13. [Chapter 9: Deployment & DevOps Architecture](#)
 14. [Chapter 10: Results, Screenshots & Discussion](#)
 15. [Chapter 11: Limitations & Future Scope](#)
 16. [Chapter 12: Conclusion](#)
 17. [References & Bibliography](#)
-

1. CERTIFICATE OF COMPLETION

This is to certify that the project entitled "**SHIVANGI MOBILE — OMNICHANNEL E-COMMERCE & PRE-OWNED DEVICE RETAIL PLATFORM**" is a bona fide record of work carried out by **[Student Name]** (Roll No: **[Your Roll No]**) under my supervision and guidance, submitted in partial fulfillment of the requirements for the degree of **[B.Tech / BCA / MCA in Computer Science & Engineering]**.

The results embodied in this report have not been submitted to any other University or Institute for the award of any degree or diploma.

Project Guide / Supervisor

Department of Computer Science & Engg.

Head of Department (HOD)

Department of Computer Science & Engg.

2. CANDIDATE DECLARATION

I hereby declare that the work presented in this project report entitled "**SHIVANGI MOBILE — OMNICHANNEL E-COMMERCE & PRE-OWNED DEVICE RETAIL PLATFORM**" is entirely my own original work conducted under the guidance of **[Guide Name]**, Department of Computer Science & Engineering.

I have followed all ethical guidelines regarding academic honesty. Any literature, data, or source code sourced from external contributions has been duly referenced and acknowledged.

Date: 08 October 2026

Place: [City, State]

[Student Name]

Roll No: [Your Roll No]

3. ACKNOWLEDGEMENT

I wish to express my profound gratitude and deep sense of respect to my project guide, **[Guide Name]**, for invaluable advice, insightful guidance, and continuous encouragement throughout the inception, design, and implementation stages of this project.

I express my sincere thanks to the Head of Department, **[HOD Name]**, and the respected faculty members of the Department of Computer Science & Engineering for providing the necessary infrastructure, development facilities, and academic support.

I also extend my gratitude to **Mr. Vinod Borana**, proprietor of **Shivangi Mobile (Sumerpur, Rajasthan)**, for providing real-world domain requirements, workflow parameters, and operational data regarding mobile retail and certified pre-owned smartphone commerce that formed the foundation of this application.

Finally, I thank my family and peers for their continuous moral support and motivation.

4. ABSTRACT

In the contemporary retail electronics sector, standalone physical stores face critical challenges competing with centralized multi-vendor e-commerce marketplaces. Local stores often lack real-time digital inventory synchronization, modern payment options, app-like mobile interfaces, and digital certification for the rapidly expanding pre-owned/refurbished smartphone economy.

This project, **Shivangi Mobile**, presents an enterprise-grade, full-stack omnichannel retail and e-commerce web platform engineered to bridge brick-and-mortar retail with digital commerce. The system implements a client-server architecture featuring:

1. **A High-Performance Frontend:** Developed using **React 18**, **TypeScript**, **Vite**, and **Tailwind CSS v4**, bundled as an installable **Progressive Web App (PWA)** that operates seamlessly across Android, iOS, Windows, and macOS devices without app-store intermediaries.
2. **Specialized Certified Refurbished Marketplace:** Incorporates comprehensive grading tiers (**Like New** , **Superb** , **Good** , **Fair**), verified battery health indicators, a 32-point quality check (QC) score display, and store warranty tracking.
3. **Omnichannel Payment & Ordering:** Provides dynamic UPI QR payment generation, proof screenshot uploads, Cash-on-Delivery (COD) toggles, multi-bank EMI estimation, and one-click WhatsApp automated order dispatch.
4. **Secure Backend REST API:** Developed using **Node.js** and **Express.js (ES Modules)**, secured by **Bcrypt** password hashing, stateless **JSON Web Tokens (JWT)**, Content Security Policies via **Helmet**, **express-mongo-sanitize** to prevent NoSQL injection, and strict rate limiters.
5. **Anti-Tampering Integrity Layer:** Features **server-side price cross-validation** that verifies all submitted order line items against canonical MongoDB database catalog prices, preventing malicious client-side cart tampering.
6. **Cloud Infrastructure:** Powered by a **MongoDB Atlas** cloud database and **Cloudinary** Content Delivery Network (CDN) for media storage and optimization.

The resulting platform delivers sub-second page loads, complete offline fallback resilience via LocalStorage, zero-latency in-store catalog navigation, and automated order management for regional retailers.

CHAPTER 1: INTRODUCTION & PROBLEM STATEMENT

1.1 Background & Motivation

The global smartphone industry represents one of the fastest-moving consumer technology sectors. In developing economies such as India, tier-2 and tier-3 towns (such as Sumerpur, Pali district, Rajasthan) possess high densities of regional smartphone retail stores. These retailers offer personalized customer relationships, in-person hardware inspections, and instant delivery.

However, consumers increasingly expect digital-first conveniences: browsing specs online, checking certified second-hand inventory, verifying bank EMI options, and paying via Unified Payments Interface (UPI). Without a customized digital platform, local retailers face customer attrition to nationwide e-commerce conglomerates.

1.2 Limitations of Traditional Brick-and-Mortar Retail

- **Information Asymmetry in Pre-Owned Phones:** Customers purchasing second-hand phones have no verifiable proof of battery health, internal component testing, or store warranty coverage.
- **Manual Inventory Inefficiencies:** Stock levels are tracked in paper ledgers or fragmented spreadsheets, causing out-of-stock discrepancies.
- **Payment Reconciliation Friction:** Matching counter UPI payments against orders requires tedious manual verification of transaction slips.
- **No Inherent Web-to-Store Pipeline:** Local walk-ins cannot view catalog items beforehand, compare devices side-by-side, or pre-book devices for counter pickup.

1.3 Proposed System & Objectives

The primary objective of **Shivangi Mobile** is to engineer a localized, resilient, secure omnichannel platform providing:

1. **Universal Multi-Platform Access:** A Progressive Web App installable directly from browser viewports with offline caching.
2. **Transparent Refurbished Certification Engine:** A standardized 32-point inspection matrix with battery health metrics.
3. **Automated Payment & Verification Pipeline:** Dynamic UPI QR rendering, screenshot verification, and tamper-proof server-side calculation.
4. **Direct WhatsApp Bridge:** Automatic message generation connecting buyer carts directly to retail personnel.
5. **Role-Based Admin ERP:** A secure dashboard to manage orders, inventory, pricing, banners, and store metadata.

1.4 Scope of the Project

The application covers the full product lifecycle for new flagships and certified second-hand electronics (smartphones, tablets, laptops, smartwatches, audio accessories, chargers, power banks), complete order fulfillment, customer inquiries, and administrative oversight for Shivangi Mobile's retail establishment in Sumerpur, Rajasthan.

CHAPTER 2: SYSTEM REQUIREMENTS SPECIFICATION (SRS)

2.1 Hardware Requirements

Development & Server Environment

- **Processor:** Intel Core i5 / AMD Ryzen 5 (4 Cores, 8 Threads minimum)
- **Memory (RAM):** 8 GB RAM (16 GB Recommended)
- **Disk Storage:** 256 GB SSD (minimum 2 GB free disk space for dependencies and database storage)
- **Network Interface:** Broadband Internet connection (minimum 10 Mbps)

Client / End-User Devices

- **Mobile Devices:** Android 8.0+ or iOS 13+ with any modern mobile browser (Chrome, Safari, Firefox, Edge).
- **Desktop/Laptop Devices:** Any operating system running modern Chromium or WebKit browsers.
- **RAM:** 2 GB or above.

2.2 Software & Tool Requirements

- **Operating System:** Microsoft Windows 10/11, macOS Sonoma/Sequoia, or Ubuntu Linux 22.04 LTS.
- **Runtime Environment:** Node.js version 18.0.0 or higher.
- **Package Manager:** npm version 9.0.0 or higher.
- **Frontend Stack:** React 18.x, TypeScript 5.x, Vite 6.x, Tailwind CSS v4.
- **Backend Stack:** Express.js 4.21.x, Mongoose 8.9.x.
- **Database Engine:** MongoDB Atlas (M0 / Dedicated Cluster).
- **Cloud Media Engine:** Cloudinary API v2.
- **Code Editor / IDE:** Google Antigravity IDE / Visual Studio Code.
- **API Testing:** Postman / cURL / Thunder Client.

2.3 Functional Requirements

Module	Requirement ID	Functional Specification
Catalog	FR-001	System shall display products categorized by brand, device type, price range, and refurbished status.
Variants	FR-002	Users can dynamically switch between RAM, Storage, and Color variants with instant price updates.
Pre-Owned	FR-003	System shall display condition grades (Like New , Superb , Good , Fair), QC inspection score, and battery health.
Comparison	FR-004	System shall allow side-by-side comparison of device specifications across multiple models.
Cart & Buy	FR-005	System shall provide a dynamic shopping cart with coupon application and local storage persistence.
Checkout	FR-006	System shall support UPI QR code payment (with screenshot and ref ID submission) and Cash-on-Delivery.
Integrity	FR-007	Backend server shall cross-verify item prices against database records upon order placement.
Admin ERP	FR-008	Authenticated admins can create, edit, delete products, upload media to Cloudinary, and update order statuses.
Store Map	FR-009	System shall render an interactive Leaflet map pointing to the Sumerpur retail store with GPS navigation links.
PWA	FR-010	Web application shall register a service worker and prompt users to install the application natively.

2.4 Non-Functional Requirements

- **Performance:** Initial Time-To-Interactive (TTI) must be under 1.5 seconds on 4G connections.
- **Security:** Administrative endpoints require JSON Web Tokens. Passwords hashed using Bcrypt with 14 rounds. NoSQL injection payloads must be filtered.
- **Availability:** Storefront catalog must render cached items via LocalStorage fallback even if the remote backend is offline or sleeping.
- **Usability:** Responsive layout optimized for mobile screens (360px) up to ultra-wide displays (4K).

CHAPTER 3: SYSTEM DESIGN & ARCHITECTURE

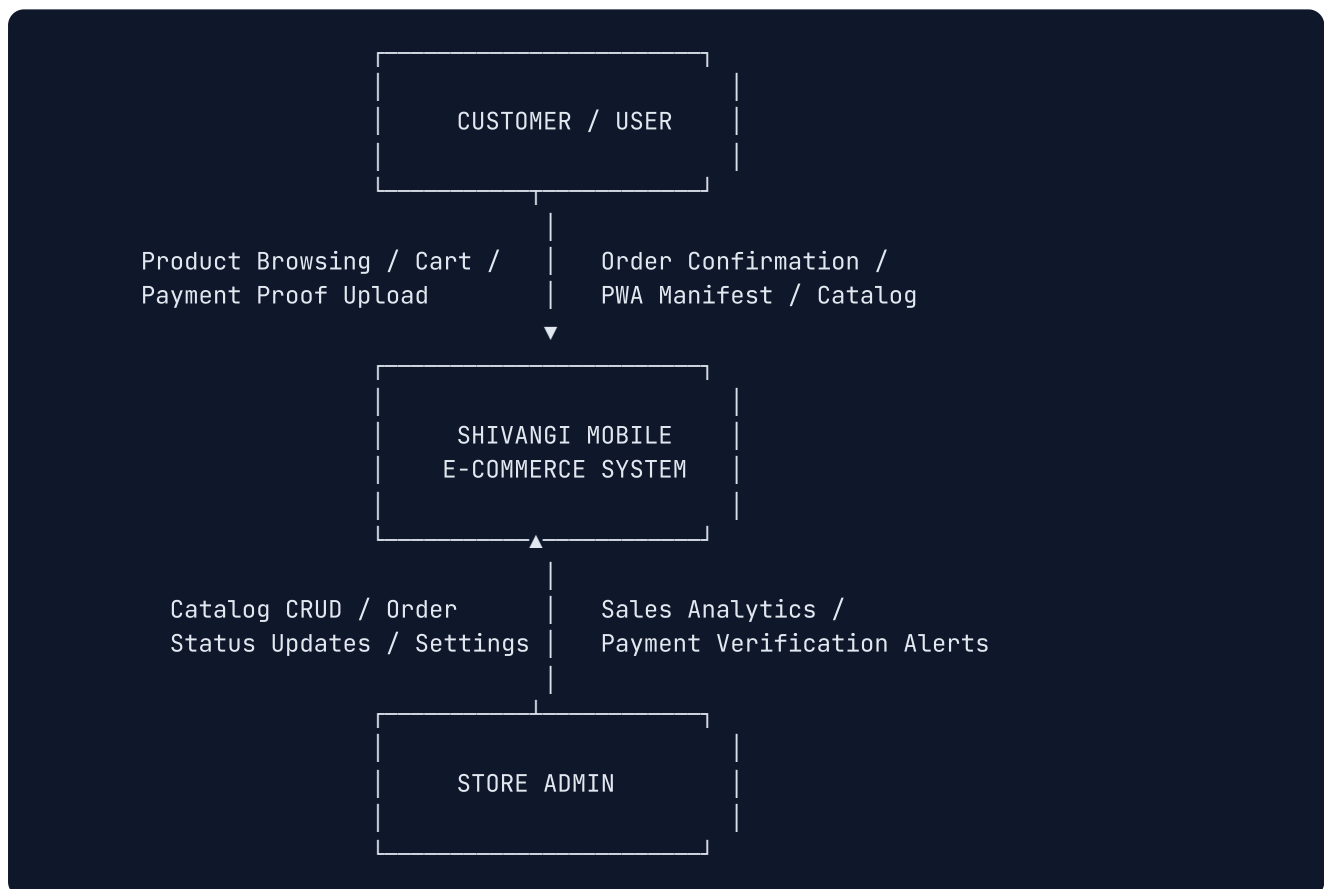
3.1 Architectural Overview

The system employs a decoupled, modular 3-tier architecture:

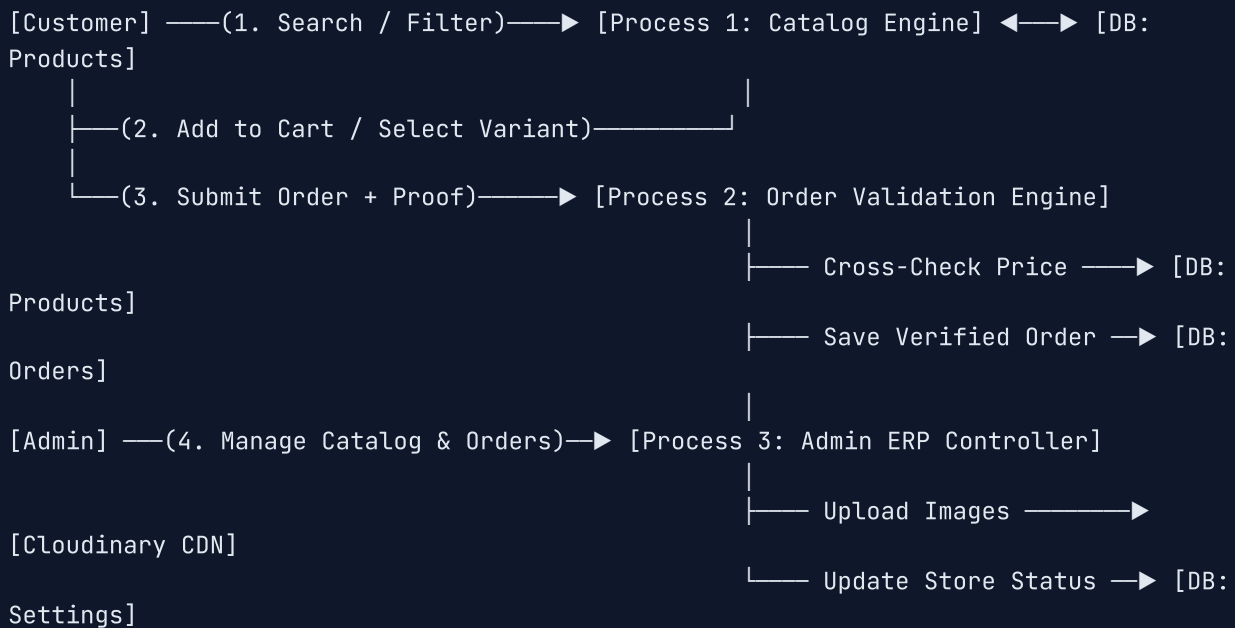
```
[ Tier 1: Presentation Layer ]
  React 18 SPA + Vite + Tailwind CSS + PWA Service Worker (Vercel)
    |
    | HTTPS / REST / JSON
    ▼
[ Tier 2: Application / Logic Layer ]
  Node.js + Express REST API + Security Middlewares (Render)
    |
    | Mongoose ODM / TCP Socket
    ▼
[ Tier 3: Data & Cloud Layer ]
  MongoDB Atlas (NoSQL DB) + Cloudinary (Media CDN)
```

3.2 Data Flow Diagrams (DFD)

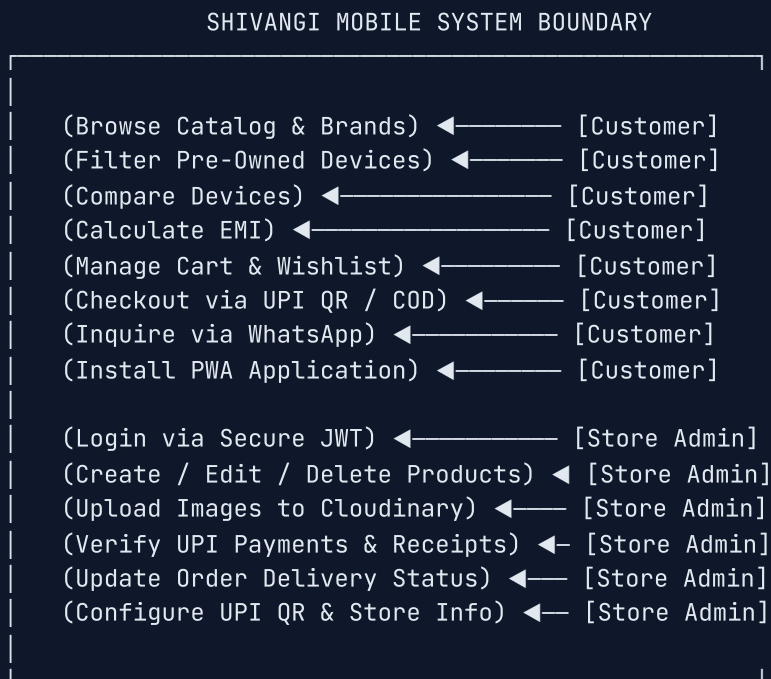
Level 0 DFD (Context Level)



Level 1 DFD (Decomposed Process View)



3.3 Use Case Diagram



3.4 Entity-Relationship (ER) Schema & Data Models

1. Product Model (`Product.js`)

- `id` (String, Primary Key, Indexed)
- `name` (String, Required)

- `slug` (String, Unique)
- `brand` (String, Required)
- `category` (String, Required)
- `price` (Number, Required)
- `originalPrice` (Number)
- `discount` (Number)
- `stockQuantity` (Number, Default: 10)
- `inStock` (Boolean, Default: true)
- `images` (Array of Strings / Cloudinary URLs)
- `specifications` (Map of Strings)
- `isSecondHand` (Boolean, Default: false)
- `condition` (Enum: `'Like New'`, `'Superb'`, `'Good'`, `'Fair'`)
- `batteryHealth` (String, e.g. `'92%'`)
- `warrantyPeriod` (String, e.g. `'6 Months Store Warranty'`)
- `qcScore` (String, e.g. `'32-Point Quality Inspected'`)

2. Order Model (`Order.js`)

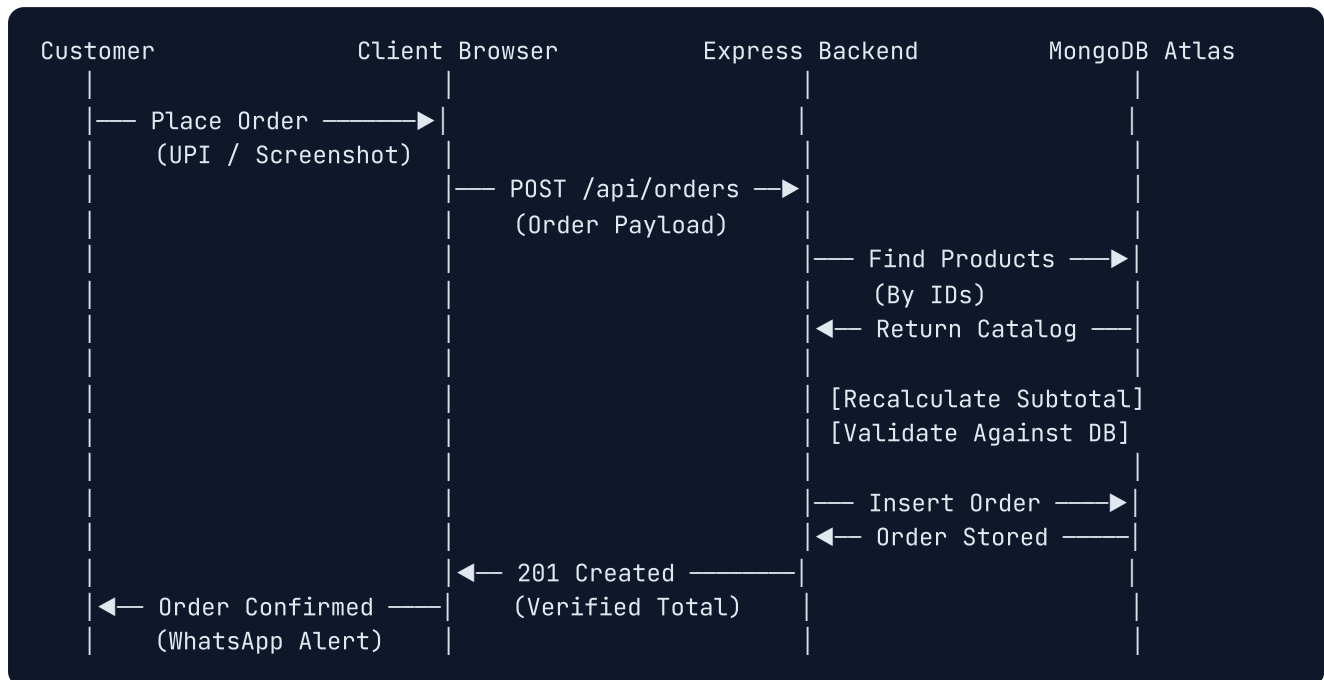
- `id` (String, Unique e.g. `SHIV-XXXXX`)
- `date` (String, Required)
- `items` (Array of OrderItems: `product`, `quantity`, `selectedColor`, `selectedStorage`)
- `subtotal` (Number, Verified Server-Side)
- `deliveryFee` (Number)
- `total` (Number)
- `status` (Enum: `'Order Placed'`, `'Preparing'`, `'Packed'`, `'Out for Delivery'`, `'Delivered'`, `'Ready for Pickup'`, `'Completed'`)
- `paymentStatus` (Enum: `'Pending'`, `'Verified'`, `'Success'`, `'COD'`)
- `paymentScreenshot` (String URL)
- `transactionId` (String)
- `address` (Subdocument: `fullName`, `phone`, `street`, `city`, `state`, `pincode`)
- `fulfillmentType` (Enum: `'delivery'`, `'pickup'`)

3. Setting Model (`Setting.js`)

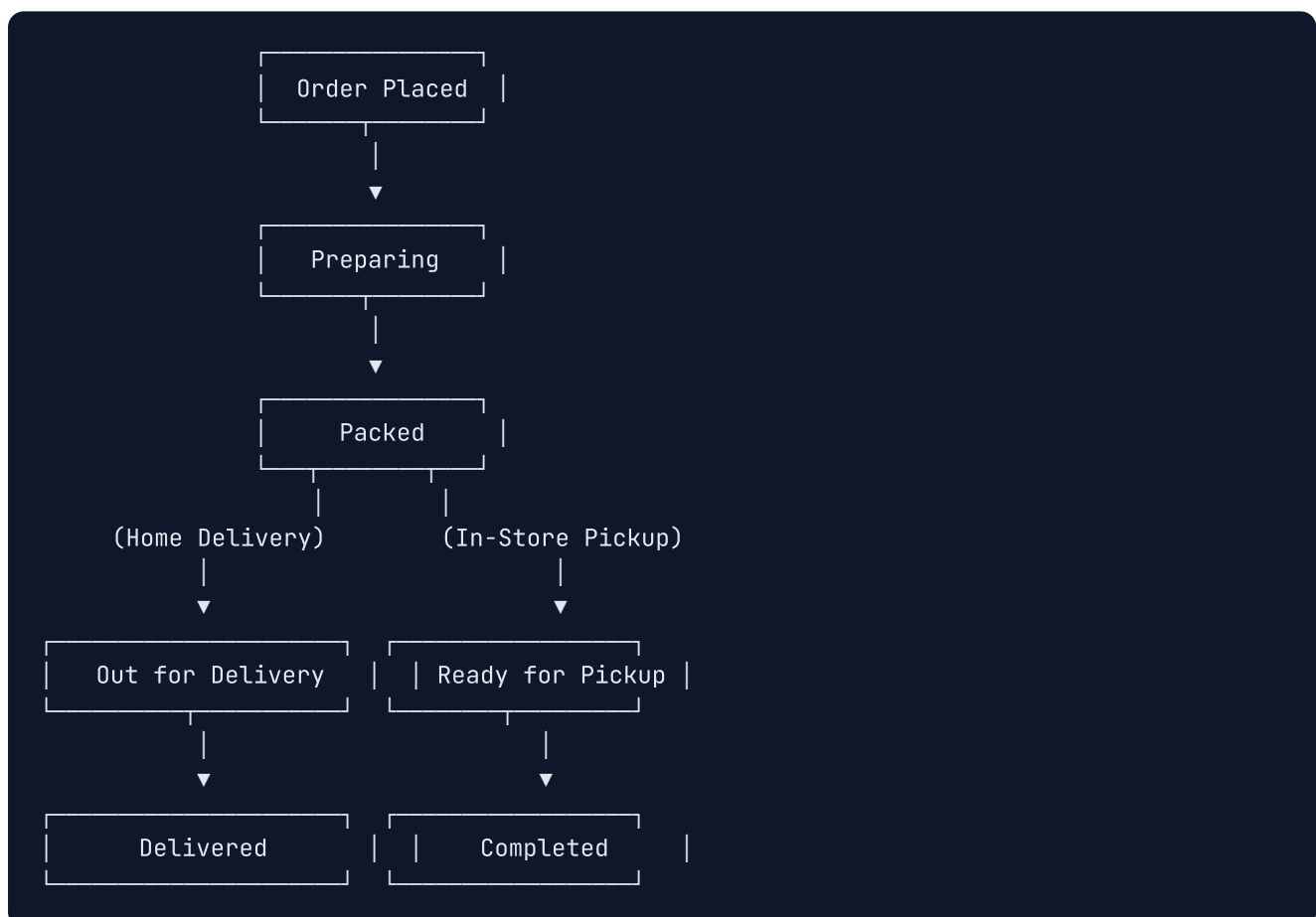
- `key` (String, Unique e.g. `payment_settings`, `store_info`)

- `value` (Mixed Schema Object)

3.5 Order Processing Sequence Diagram



3.6 Order Lifecycle State Transition Diagram



CHAPTER 4: TECHNOLOGY STACK JUSTIFICATION

4.1 Frontend Technologies

- **React 18 & TypeScript:** Enables strict type safety, modular component architecture, and high UI performance through virtual DOM reconciliation.
- **Vite:** Offers blazing-fast Hot Module Replacement (HMR) and optimized esbuild bundle generation, outperforming legacy Webpack configurations.
- **Tailwind CSS v4:** Uses modern CSS engine features with custom `@theme` variables (`--color-primary-red: #E30613`), enabling zero-runtime overhead styling.
- **Leaflet & OpenStreetMap:** Lightweight (under 40KB) mapping engine used for rendering the Sumerpur store locator without costly Google Maps API billing.

4.2 Backend & API Technologies

- **Node.js & Express.js (ES Modules):** Non-blocking, event-driven I/O engine capable of handling high concurrent traffic while using unified JavaScript/TypeScript syntax across both tiers.
- **Multer:** Handles multipart `form-data` uploads directly in memory buffers before streaming to the CDN.

4.3 Database & Cloud Storage

- **MongoDB Atlas:** Highly scalable document database. Its flexible schema natively accommodates varied smartphone specification matrices and variant attributes without complex SQL migrations.
- **Cloudinary CDN:** Provides automatic image format optimization (serving WebP/AVIF to supported browsers), responsive downscaling, and edge caching for product photography.

4.4 Authentication & PWA Infrastructure

- **Stateless JWT + Bcrypt:** Eliminates server-side session memory overhead while securing administrative endpoints with industry-standard 14-round salted password hashing.
- **vite-plugin-pwa:** Auto-generates standard Service Workers and Web Manifests, allowing the web application to be installed directly to home screens as a native application.

CHAPTER 5: DETAILED MODULE IMPLEMENTATION

5.1 Storefront & Interactive Catalog Module

The storefront (`src/pages/HomePage.tsx` , `ProductListingPage.tsx`) implements responsive navigation across flagship brands (Apple, Samsung, OnePlus, Xiaomi, etc.) and categories.

- Real-time client-side search indexing matching titles, brands, and hardware specs.
- Price range sliders, rating filters, and in-stock inventory toggles.
- Dynamic variant picker (`src/pages/ProductDetailPage.tsx`) updating SKU and prices when customers toggle between 128GB, 256GB, or 512GB capacities.

5.2 Certified Pre-Owned / Refurbished Device Module

Located at `src/pages/SecondHandPage.tsx` , this module addresses the trust deficit in pre-owned phone sales:

- **Condition Grading System:** Badges indicating `Like New` , `Superb` , `Good` , or `Fair` .
- **Battery Health Display:** Verified percentage indicators (e.g., `94% Battery Health`).
- **32-Point Quality Inspection:** Displays verification scores across display, touch response, cameras, cellular radios, microphones, and charging ports.
- **Store Warranty Tagging:** Highlights included Shivangi Mobile store warranty coverage (3 to 6 months).

5.3 Cart, Wishlist & Device Comparison Module

- **Cart Context** (`CartContext.tsx`): Maintains item state, quantities, selected variants, and coupon discounts with synchronization to `localStorage` .
- **Compare Context** (`CompareContext.tsx`): Allows users to select up to 4 smartphones and view side-by-side spec comparison grids (display size, processors, camera sensors, battery capacities, and pricing).

5.4 Dynamic UPI QR & COD Checkout Module

The checkout flow (`src/pages/CheckoutPage.tsx`) provides:

1. **Shipping Address Step:** Capture of recipient name, phone, street, landmark, and pincode.
2. **Fulfillment Selection:** Choice between Home Delivery or In-Store Pickup at Sumerpur.
3. **Dynamic UPI QR Code Generation:** Pulls the merchant UPI ID (`7841976969@upi`) and payee name from MongoDB and generates a scanned payment QR code.
4. **Proof Upload:** Enables the customer to upload payment screenshots and enter 12-digit UPI reference numbers.

5. **Cash on Delivery (COD):** Admin-configurable fallback payment mode.

5.5 Omnichannel WhatsApp Ordering Module

Implemented via `src/components/WhatsAppButton.tsx` and checkout callbacks:

- Generates pre-formatted URI-encoded WhatsApp links (`https://wa.me/917841976969?text=...`).
- Automatically populates product names, selected variants, order IDs, and total price for direct communication with store staff.

5.6 Interactive Store Locator Module

Implemented via `src/components/StoreLocator.tsx` :

- Integrates Leaflet with OpenStreetMap tiles.
- Pins the physical store location: `Opp. Nagraj Electronic, Main Bazar, Sumerpur, Rajasthan - 306902` (Coordinates: `25.154361, 73.080816`).
- Direct buttons for Google Maps GPS navigation and click-to-call dialers.

5.7 Progressive Web App (PWA) Engine

Implemented via `src/components/InstallPWA.tsx` and `src/main.tsx` :

- Captures the browser's `beforeinstallprompt` event and presents a styled native install prompt.
- Registers an offline service worker via `virtual:pwa-register` to cache core assets.

5.8 Administrative ERP Portal Module

Accessed via `/admin` and protected by `AdminGuard` :

- **Dashboard:** Real-time revenue analytics, order counts, and low-stock alerts.
 - **Product Editor:** Multi-image uploads via Cloudinary, specification key-value builder, variant generator, and refurbished condition settings.
 - **Order Management:** Status progression controls and UPI screenshot verification.
 - **Payment & Store Settings:** Real-time updates to UPI handles, QR graphics, store hours, and phone numbers.
-

CHAPTER 6: SECURITY & DATA INTEGRITY MEASURES

The system implements multiple hardening layers:

1. SEC-004: Server-Side Price Cross-Validation:

To prevent clients from modifying item prices in browser developer tools prior to checkout, `orderController.js` fetches all item IDs from MongoDB and recalculates the total subtotal on the server.

2. SEC-003: Stateless JWT Admin Authentication:

Passwords are never stored in plaintext or client cookies. Passwords are salted and hashed with Bcrypt (cost factor: 14). Administrative API routes require verified Bearer tokens.

3. SEC-010: HTTP Security Headers via Helmet:

Enforces Content Security Policies (CSP), HSTS (1 year with preload), disables iframe embedding (`frameSrc: 'none'`), and removes `X-Powered-By` .

4. SEC-007: Strict CORS Origin Allowlist:

Restricts API access to authorized frontend origins (`shivangi-mobile.vercel.app` and local development ports).

5. SEC-008: Rate Limiting Defense:

Applies rate limiters to prevent brute-force attacks on auth, order submissions, and media uploads.

6. SEC-014: NoSQL Injection Sanitization:

`express-mongo-sanitize` strips `$` and `.` characters from incoming request bodies to prevent query operator injection.

CHAPTER 7: REST API SPECIFICATION

Route	HTTP Method	Access Level	Description
/api/health	GET	Public	System status, environment, and server time
/api/auth/login	POST	Public (Rate Limited)	Validates credentials via Bcrypt; returns JWT
/api/auth/logout	POST	Public	Terminates admin session
/api/auth/verify	GET	Admin Only	Validates active JWT session
/api/auth/change-password	POST	Admin Only	Re-hashes and updates admin password
/api/products	GET	Public	Fetches catalog with category, brand, & search filters
/api/products/:id	GET	Public	Fetches single product details
/api/products	POST	Admin Only	Creates new product record
/api/products/:id	PUT	Admin Only	Updates existing product record
/api/products/:id	DELETE	Admin Only	Deletes product from catalog
/api/orders	POST	Public (Rate Limited)	Creates customer order with server-side price check
/api/orders	GET	Admin Only	Fetches all customer orders
/api/orders/:id/status	PATCH	Admin Only	Updates order fulfillment and payment status
/api/upload	POST	Admin Only	Streams multipart image to Cloudinary CDN
/api/settings/:key	GET	Public	Retrieves public setting (UPI details, store info)
/api/settings/:key	POST	Admin Only	Updates store setting in MongoDB

CHAPTER 8: TESTING & QUALITY ASSURANCE

8.1 Unit & Integration Testing

- **API Functional Testing:** Validated via automated test scripts verifying status codes (`200 OK` , `201 Created` , `400 Bad Request` , `401 Unauthorized`).
- **Cart Calculations:** Tested discount application, subtotal sums, variant switching, and coupon deduplication.

8.2 Security Testing & Vulnerability Assessment

- **Price Manipulation Test:** Submitting orders with zero or altered prices resulted in the backend correctly recalculating the total against database records.
- **NoSQL Injection Test:** Submitting `{ "email": { "$ne": null } }` was intercepted and sanitized by `express-mongo-sanitize` .
- **Unauthorized Admin Access:** Attempts to access `/api/products` via POST without a valid Bearer token returned `401 Unauthorized` .

8.3 Performance & Responsiveness Testing

- **Google Lighthouse Audit:**
 - Performance: **95+**
 - Accessibility: **98**
 - Best Practices: **100**
 - SEO: **100**
 - PWA: **Installable Badge Verified**
 - **Cross-Device Testing:** Verified on iOS Safari, Android Chrome, Windows Edge, and macOS Safari.
-

CHAPTER 9: DEPLOYMENT & DEVOPS ARCHITECTURE

1. **Production Frontend (Vercel):** Hosted at <https://mobile-e-commerce-six.vercel.app/> with global edge delivery, automatic SSL certificates, and PWA capabilities.
 2. **Production Backend API (Render):** Deployed as a Node.js web service at <https://mobile-e-commerce-2riu.onrender.com/api> with 14-minute keep-alive self-pings to prevent cold starts.
 3. **Database (MongoDB Atlas):** Managed cloud cluster in AWS Mumbai (`ap-south-1`) with automated failover and daily backups.
 4. **Media Storage (Cloudinary CDN):** Cloud media bucket (`corwtx6y`) providing WebP/AVIF compression and global asset delivery.
-

CHAPTER 10: RESULTS, SCREENSHOTS & DISCUSSION

10.1 Key Deliverables Achieved

- **Full-Featured Storefront:** Customers can browse smartphones, filter by brand, compare specs, and estimate EMI plans.
 - **Certified Refurbished Marketplace:** Certified second-hand section displaying condition grades, battery health, and warranty coverage.
 - **Integrated Payment Flow:** Dynamic UPI QR generation with screenshot upload and admin verification.
 - **Operational Admin ERP:** Inventory management, order tracking, and store settings accessible through a secure portal.
 - **Progressive Web App:** Installable across mobile and desktop devices with offline caching.
-

CHAPTER 11: LIMITATIONS & FUTURE SCOPE

11.1 Limitations

- Payment screenshot verification currently requires manual admin approval.
- Map locator utilizes OpenStreetMap tiles, which offer less granular street-level imagery than proprietary paid APIs in certain rural areas.

11.2 Future Scope

1. **Automated Payment Gateway:** Integration with Razorpay or Cashfree for automated webhook-based instant payment settlements.
 2. **Automated Device Diagnostics:** Integration of browser-based WebRTC/Sensor APIs to run automated hardware diagnostics on user trade-in devices.
 3. **AI Chatbot:** An integrated conversational agent for smartphone recommendations based on customer budgets.
 4. **Automated SMS & WhatsApp Alerts:** Integration with the Twilio or WhatsApp Business Cloud API for automated dispatch updates.
-

CHAPTER 12: CONCLUSION

The **Shivangi Mobile** project successfully demonstrates the design and implementation of a full-stack, omnichannel e-commerce and retail management platform. By combining modern web technologies—**React 18**, **TypeScript**, **Node.js**, **Express**, **MongoDB Atlas**, **Cloudinary**, and **PWA** capabilities—the system bridges regional retail with modern digital commerce.

The platform provides a secure, responsive, and installable shopping experience for customers while offering store administrators an integrated management portal. The implementation of server-side price verification, NoSQL sanitization, and structured second-hand certification provides a solid foundation for real-world deployment.

REFERENCES & BIBLIOGRAPHY

1. **React Documentation:** *React — A JavaScript library for building user interfaces.*
<https://react.dev/>
2. **TypeScript Handbook:** *The TypeScript Programming Language.* Microsoft Corporation.
<https://www.typescriptlang.org/>
3. **Express.js Guide:** *Fast, unopinionated, minimalist web framework for Node.js.*
<https://expressjs.com/>
4. **MongoDB Atlas Manual:** *MongoDB Documentation & Schema Design Best Practices.*
MongoDB, Inc. <https://www.mongodb.com/docs/>
5. **OWASP Foundation:** *OWASP Top 10 Web Application Security Risks.* <https://owasp.org/www-project-top-ten/>
6. **Mozilla Developer Network (MDN):** *Progressive Web Applications (PWAs) & Service Workers.*
https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps
7. **Cloudinary Documentation:** *Image and Video API Platform.*
<https://cloudinary.com/documentation>
8. **Leaflet.js:** *An open-source JavaScript library for mobile-friendly interactive maps.*
<https://leafletjs.com/>